Faculty of Computing, Electronics and Science

TABLE OF CONTENTS

Table of Figures:

SCENARIO DESCRIPTION

BACKGROUND OF EUROPARTS.COM

EuroParts.com is a web-based stage having some expertise in the dissemination of car parts and adornments across Europe. The organization intends to give a helpful and effective way for car organizations and devotees to source quality parts for different makes and models. With many providers and a broad stock, EuroParts.com endeavors to be a one-stop answer for all auto part needs.

DATABASE TOOLS AND ENVIRONMENT

Throughout creating and examining the data set questions introduced in this report, Prophet Data set and SQL Engineer were used as the essential devices. Prophet Data set gives a strong and versatile stage for overseeing and questioning huge datasets, making it appropriate for the intricate information prerequisites of Europarts.com. SQL Designer, Prophet's incorporated improvement climate, worked with the creation and execution of SQL questions, smoothing out the most common way of interfacing with the data set.

ENTITY-RELATIONSHIP DIAGRAM (ERD)

An improved and fractional ERD for EuroParts.com is intended to delineate key elements and connections inside the framework. The ERD gives an outline of how elements like Parts, Providers, Stockrooms, Containers, Areas, Solicitations, and InvoiceDetails are interconnected. The chart grandstands the connections between these elements, assisting partners with understanding the information stream and relationship inside the EuroParts.com stage.

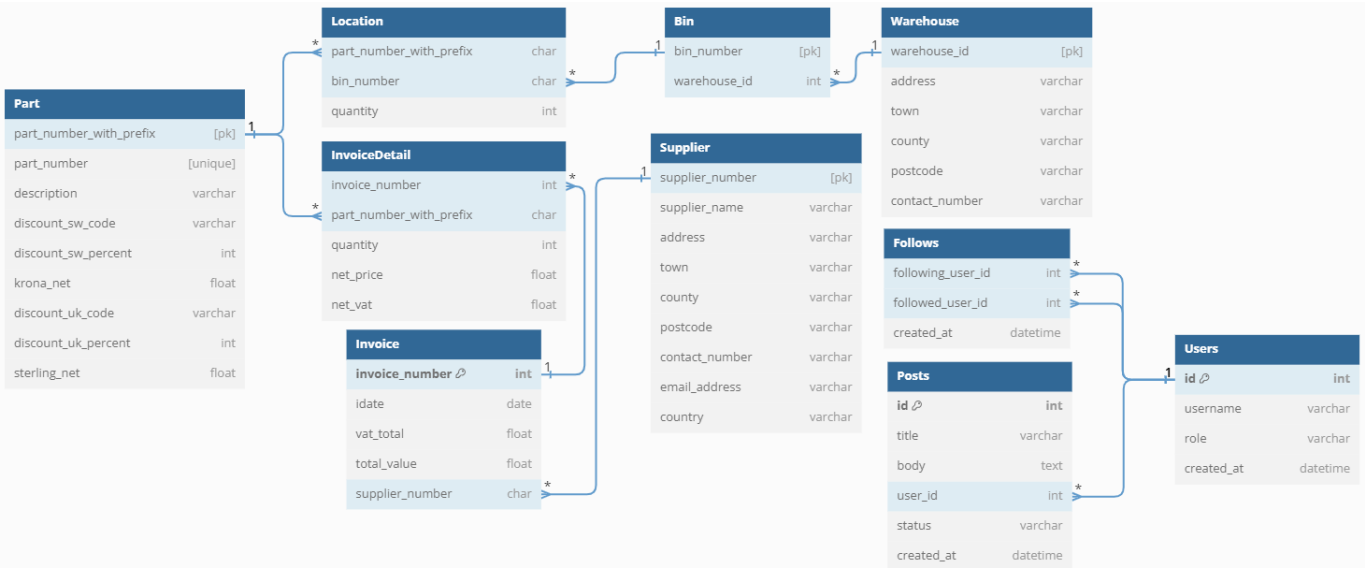
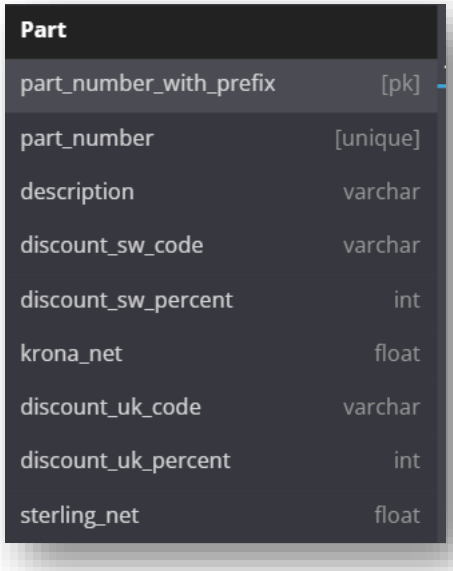


FIGURE 1: ER DIAGRAM

ENTITY DEFINITIONS/ATTRIBUTES

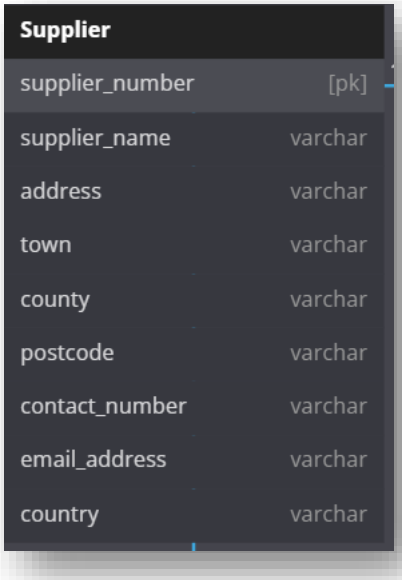
1. **Part:**

- Attributes:
    - part_number_with_prefix [Primary Key (PK)]: Interestingly recognizes a section with a prefix.
    - part_number [Unique]: Unique identifier for a part.
    - description: Describes the part.
    - discount_sw_code: Code for the SW (Sweden) market discount.
    - discount_sw_percent: Percentage of discount for the SW market.
    - krona_net: Net price in Swedish Krona.
    - discount_uk_code: Code for the UK market discount.
    - discount_uk_percent: Percentage of discount for the UK market.
    - sterling_net: Net price in British Pounds Sterling.



- Reasoning: The combination of part_number_with_prefix as the primary key ensures a unique identification of each part, while part_number is declared as unique to avoid duplication. This design accommodates the specific requirements for identifying and managing parts.

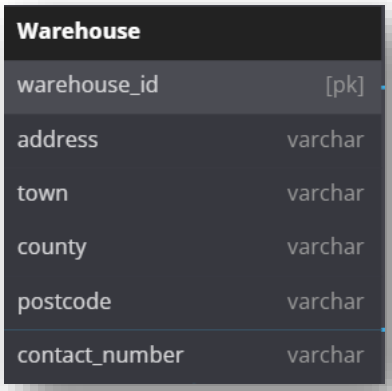
2. **Supplier:**
    - Attributes:
        - supplier_number [PK]: Unique identifier for a supplier.
        - supplier_name: Name of the supplier.
        - address: Supplier's address.
        - town: Supplier's town.
        - county: Supplier's county.
        - postcode: Supplier's postcode.
        - contact_number: Supplier's contact number.
        - email_address: Supplier's email address.
        - country: Supplier's country.



    - Reasoning: The supplier_number is designated as the primary key to uniquely identify each supplier. This key ensures efficient referencing in relationships with other entities.

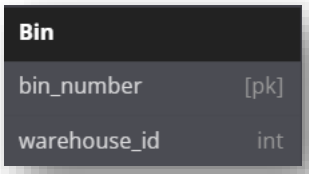
3. **Warehouse:**

- Attributes:
  - warehouse_id [PK]: Unique identifier for a warehouse.
  - address: Warehouse address.
  - town: Warehouse town.
  - county: Warehouse County.
  - postcode: Warehouse postcode.
  - contact_number: Warehouse contact number.



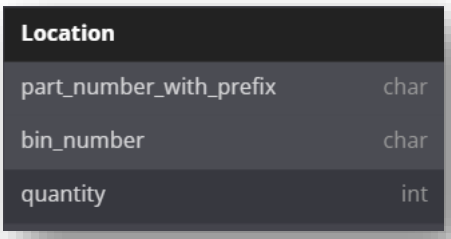
- Reasoning: The warehouse_id is chosen as the primary key for warehouses to provide a unique identifier. This key facilitates relationships with other entities, such as bins.

4. **Bin:**
   - Attributes:
     - bin_number [PK]: Unique identifier for a bin.
     - warehouse_id [FK]: Foreign key referencing the warehouse_id in the Warehouse table.



   - Reasoning: The combination of bin_number as the primary key and warehouse_id as a foreign key establishes a unique identification for each bin within the context of a warehouse. This structure ensures a clear association between bins and warehouses.

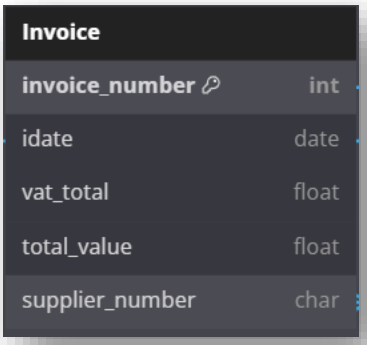
5. **Location:**
   - Attributes:
     - part_number_with_prefix [FK]: Foreign key referencing the part_number_with_prefix in the Part table.
     - bin_number [FK]: Foreign key referencing the bin_number in the Bin table.
     - quantity: Quantity of parts in the location.



   - Reasoning: The foreign keys (part_number_with_prefix and bin_number) establish relationships with the Part and Bin entities, creating a connection between parts and their physical storage locations.

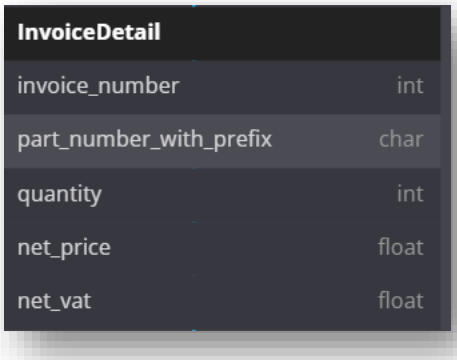
6. **Invoice:**
   - Attributes:

- invoice_number [PK]: Unique identifier for an invoice.
- idate: Date of the invoice.
- vat_total: Total VAT for the invoice.
- total_value: Total value of the invoice.
- supplier_number [FK]: Foreign key referencing the supplier_number in the Supplier table.

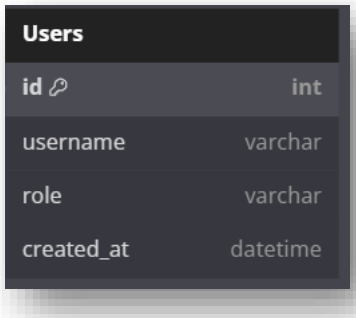


- Reasoning: The invoice_number is designated as the primary key for unique identification of each invoice. The foreign key supplier_number establishes a relationship with the Supplier entity, linking invoices to their respective suppliers.

7. **InvoiceDetail:**
   - Attributes:
     - invoice_number [FK]: Foreign key referencing the invoice_number in the Invoice table.
     - part_number_with_prefix [FK]: Foreign key referencing the part_number_with_prefix in the Part table.
     - quantity: Quantity of parts in the invoice detail.
     - net_price: Net price for the part in the invoice.
     - net_vat: Net VAT for the part in the invoice.

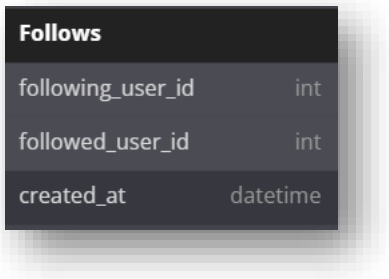


   - Reasoning: The foreign keys (invoice_number and part_number_with_prefix) create relationships with the Invoice and Part entities, establishing a connection between invoice details and the corresponding invoices and parts.
8. **Users:**
   - Attributes:
     - id [PK]: Unique identifier for a user.
     - username: User's username.
     - role: User's role (e.g., customer, administrator).
     - created_at: Timestamp indicating when the user account was created.



   - Reasoning: The id is selected as the primary key to uniquely identify users. This key is utilized in relationships with other entities, reflecting associations with users.
9. **Follows:**

- Attributes:
    - following_user_id [FK]: Foreign key referencing the id in the Users table (user who follows).
    - followed_user_id [FK]: Foreign key referencing the id in the Users table (user being followed).
    - created_at: Timestamp indicating when the follow relationship was established.



- Reasoning: The foreign keys (following_user_id and followed_user_id) establish relationships with the Users entity, creating associations between users who follow and those being followed.

10. **Posts:**

- Attributes:
    - id (Primary Key): Uniquely identifies each post.
    - title: Title of the post.
    - body: Content of the post.
    - user_id (Foreign Key): Links to the user creating the post.
    - status: Status of the post.
    - created_at: Timestamp of post creation.
- *Reasoning:*
    - id is chosen as the primary key for unique identification of posts. user_id is a foreign key linking posts to the user who created them.

11. **Countries**

- Attributes:
    - code (Primary Key): Uniquely identifies each country.
    - Other attributes describing countries.
- Reasoning: A Countries table is recommended for managing country-related information, such as currency codes, exchange rates, etc. The code attribute serves as the primary key for unique identification.

TABLE DEFINITIONS IN SQL (DDL)

In the provided SQL code, several tables have been defined to structure and organize data within a relational database. Each table serves a specific purpose in the context of a system or application. Let's delve into an explanation of the table definitions:

Part Table: The "Part" table is designed to store information about various parts, each identified by a unique part number with a prefix. It incorporates subtleties, for example, the portrayal, rebate codes, estimating data in both Krona and Authentic, shaping a thorough portrayal of the parts in the framework.

Supplier Table: The " "Provider" table spotlights on catching data connected with providers, for example, their name, address, contact subtleties, and country. The provider number fills in as the essential key, guaranteeing every provider is particularly recognizable inside the data set.

Warehouse Table: The "Distribution center" table characterizes the properties of various stockrooms, including their area subtleties and contact data. The stockroom ID goes about as the essential key, extraordinarily recognizing each distribution center passage.

Bin Table: The "Container" table partners canister numbers with explicit distribution centers, laying out an association between the actual stockpiling areas and the actual stockrooms. Each receptacle number is remarkable and fills in as the essential key for this table.

Location Table: The "Area" table lays out a connection among parts and containers, showing the amount of a specific part put away in a particular receptacle. This table is crucial for stock administration, with a composite essential key framed by the part number with prefix and container number.

Invoice Table: The "Receipt" table records data about solicitations, including receipt number, date, Tank absolute, all out esteem, and the provider related with the exchange. The receipt number fills in as the essential key, guaranteeing the uniqueness of each receipt passage.

InvoiceDetail Table: The "InvoiceDetail" table holds point by point data about individual parts inside a receipt, including amount, net cost, and net Tank. It lays out a connection among solicitations and parts, framing a composite essential key utilizing the receipt number and part number with prefix.

InvoiceDetail" table holds point by point data about individual parts inside a receipt, including amount, net cost, and net Tank. It lays out a connection among solicitations and parts, framing a composite essential key utilizing the receipt number and part number with prefix.

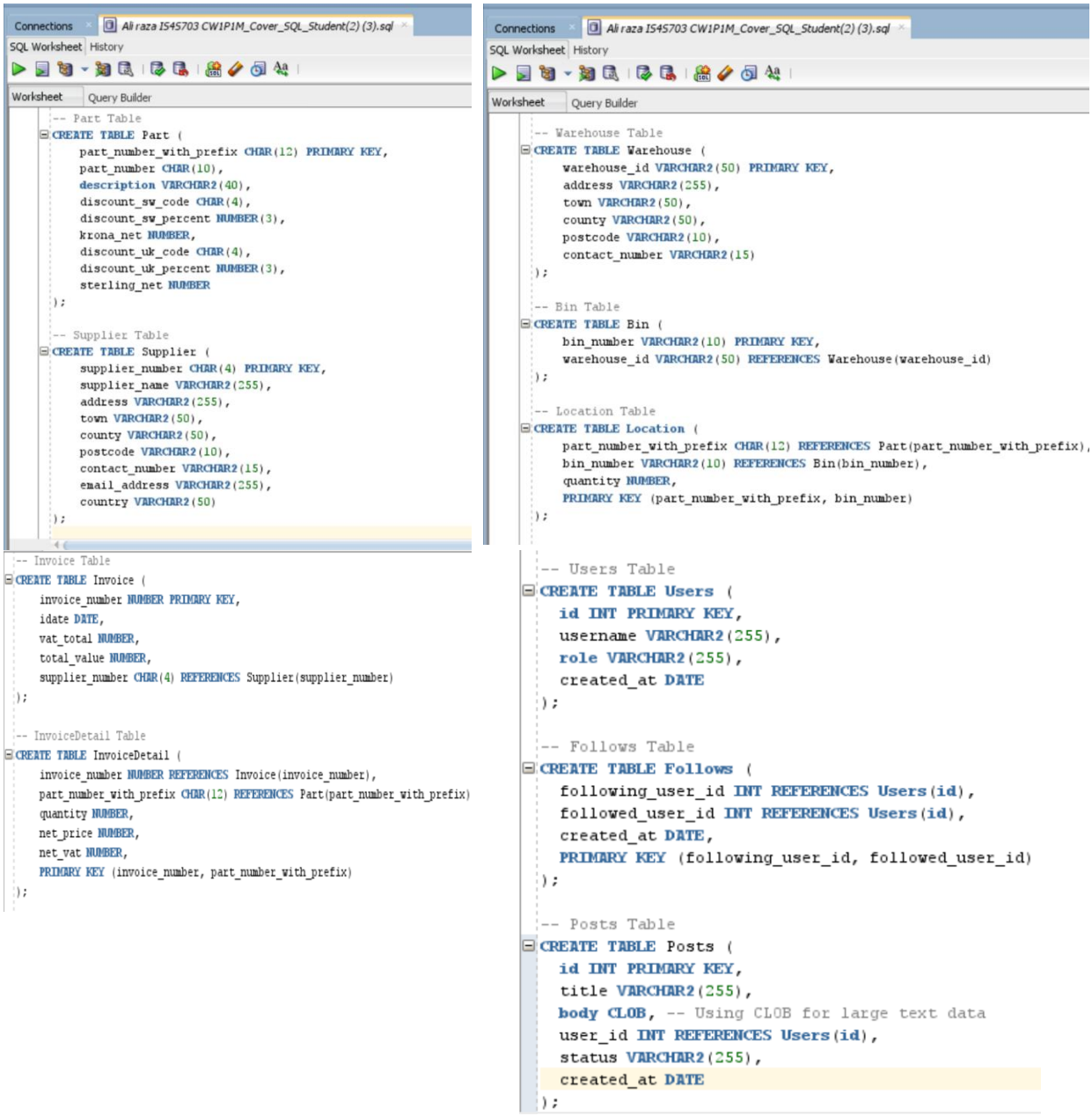
Users Table: : The "Clients" table is intended to oversee client data, including an ID, username, job, and creation date. The client ID goes about as the essential key, particularly distinguishing every client in the framework.

Follows Table: The "Follows" table lays out a connection between clients, following devotees and followed clients. The blend of following_user_id and followed_user_id structures a composite essential key, guaranteeing the uniqueness of each follow relationship.

Posts Table: The "Posts" table handles the capacity of post-related data, including title, body, client ID, status, and creation date. The post ID fills in as the essential key, remarkably recognizing each post section in the data set.

In rundown, these table definitions all in all structure a very much organized social data set blueprint for overseeing parts, providers, stockrooms, containers, areas, solicitations, receipt subtleties, clients,

follows, and posts. The connections laid out between tables add to viable information association and recovery inside the predefined framework.



```
-- Part Table
CREATE TABLE Part (
    part_number_with_prefix CHAR(12) PRIMARY KEY,
    part_number CHAR(10),
    description VARCHAR2(40),
    discount_sw_code CHAR(4),
    discount_sw_percent NUMBER(3),
    krona_net NUMBER,
    discount_uk_code CHAR(4),
    discount_uk_percent NUMBER(3),
    sterling_net NUMBER
);

-- Supplier Table
CREATE TABLE Supplier (
    supplier_number CHAR(4) PRIMARY KEY,
    supplier_name VARCHAR2(255),
    address VARCHAR2(255),
    town VARCHAR2(50),
    county VARCHAR2(50),
    postcode VARCHAR2(10),
    contact_number VARCHAR2(15),
    email_address VARCHAR2(255),
    country VARCHAR2(50)
);
```

```
-- Warehouse Table
CREATE TABLE Warehouse (
    warehouse_id VARCHAR2(50) PRIMARY KEY,
    address VARCHAR2(255),
    town VARCHAR2(50),
    county VARCHAR2(50),
    postcode VARCHAR2(10),
    contact_number VARCHAR2(15)
);

-- Bin Table
CREATE TABLE Bin (
    bin_number VARCHAR2(10) PRIMARY KEY,
    warehouse_id VARCHAR2(50) REFERENCES Warehouse(warehouse_id)
);

-- Location Table
CREATE TABLE Location (
    part_number_with_prefix CHAR(12) REFERENCES Part(part_number_with_prefix),
    bin_number VARCHAR2(10) REFERENCES Bin(bin_number),
    quantity NUMBER,
    PRIMARY KEY (part_number_with_prefix, bin_number)
);
```

```
-- Invoice Table
CREATE TABLE Invoice (
    invoice_number NUMBER PRIMARY KEY,
    idate DATE,
    vat_total NUMBER,
    total_value NUMBER,
    supplier_number CHAR(4) REFERENCES Supplier(supplier_number)
);

-- InvoiceDetail Table
CREATE TABLE InvoiceDetail (
    invoice_number NUMBER REFERENCES Invoice(invoice_number),
    part_number_with_prefix CHAR(12) REFERENCES Part(part_number_with_prefix)
    quantity NUMBER,
    net_price NUMBER,
    net_vat NUMBER,
    PRIMARY KEY (invoice_number, part_number_with_prefix)
);
```

```
-- Users Table
CREATE TABLE Users (
  id INT PRIMARY KEY,
  username VARCHAR2(255),
  role VARCHAR2(255),
  created_at DATE
);

-- Follows Table
CREATE TABLE Follows (
  following_user_id INT REFERENCES Users(id),
  followed_user_id INT REFERENCES Users(id),
  created_at DATE,
  PRIMARY KEY (following_user_id, followed_user_id)
);

-- Posts Table
CREATE TABLE Posts (
  id INT PRIMARY KEY,
  title VARCHAR2(255),
  body CLOB, -- Using CLOB for large text data
  user_id INT REFERENCES Users(id),
  status VARCHAR2(255),
  created_at DATE
);
```

FIGURE 2:TABLES CREATIONS SUCCESSFULLY

SQL DML STATEMENTS

The SQL Information Control Language (DML) proclamations in the gave code perform different tasks, including information addition into tables. The beneath given are instances of information populate in each table:

```
-- Insert data into Part table
```

```
INSERT INTO Part (part_number_with_prefix, part_number, description, discount_sw_code,
discount_sw_percent, krona_net, discount_uk_code, discount_uk_percent, sterling_net)
VALUES ('104531331', '4531331', 'Bushing', 'D51', 35, 38.81, '1', 25, 3.68);
```

```
-- Inserting data into the Supplier table
INSERT INTO Supplier (supplier_number, supplier_name, address, town, county, postcode,
contact_number, email_address, country)
VALUES ('H001', 'Higher Oak', 'Oak Road, Wrexham Ind Est', 'Wrexham', 'Wrexham', 'LL13
9RG', '01443456123', 'HighOak@hotmail.co.uk', 'Wales');
```

```
-- Inserting data into the Warehouse table
INSERT INTO Warehouse (warehouse_id, address, town, county, postcode, contact_number)
VALUES ('Steve Haverfordwest', 'Riverview, Spittal', 'Haverfordwest', 'Pembrokeshire', 'SA62
3BA', '01437 123456');
```

```
-- Inserting data into the Bin table
INSERT INTO Bin (bin_number, warehouse_id)
VALUES ('A2L', 'Steve Haverfordwest');
```

```
-- Inserting data into the Location table
INSERT INTO Location (part_number_with_prefix, bin_number, quantity)
VALUES ('104531331', 'O137', 25);
```

```
-- Inserting data into the Invoice table
INSERT INTO Invoice (invoice_number, idate, vat_total, total_value, supplier_number)
VALUES ('100001', TO_DATE('01-NOV-23', 'DD-MON-YY'), 1.42, 8.1, 'N001');
```

```
-- Inserting data into the InvoiceDetail table
INSERT INTO InvoiceDetail (invoice_number, part_number_with_prefix, quantity, net_price,
net_vat)
VALUES ('100001', '104531331', 2, 7.36, 1.29);
```

QUERIES

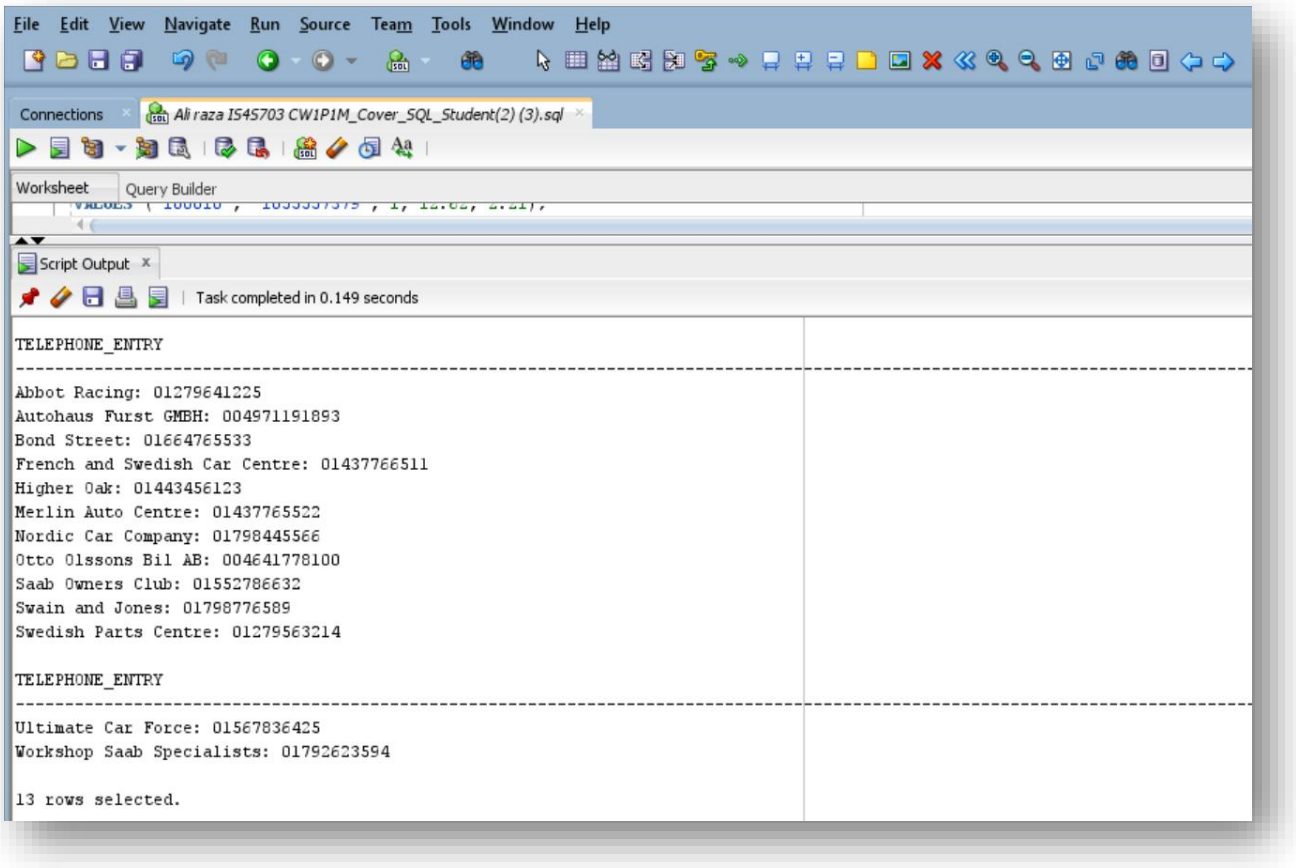
QUERY 1: TELEPHONE LIST FOR SUPPLIERS

In this SQL question, I made a view named SupplierTelephoneList to create a phone list for providers. The view is built by linking the provider's name and contact number, isolated by a colon, for every provider in the Provider table. The view is requested sequentially founded on the provider names in rising request.

To inquiry the phone list, the SELECT assertion is utilized to bring all passages from the SupplierTelephoneList view. This inquiry gives a thorough perspective on provider contact data, making it an important device for clients who need fast admittance to provider phone subtleties. The production of such perspectives improves the ease of use and association of the data set by introducing data in an unmistakable and organized way, adding to a more proficient work process for clients collaborating with provider information.

```
-- Create a View for Telephone List
CREATE VIEW SupplierTelephoneList AS
```

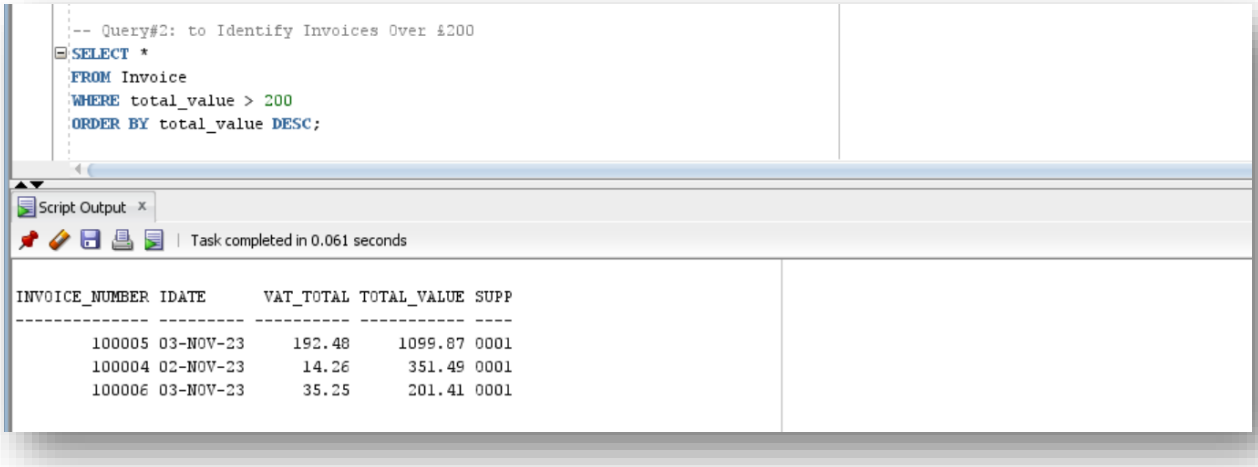
```
SELECT supplier_name || ': ' || contact_number AS telephone_entry
FROM Supplier
ORDER BY supplier_name ASC;
-- Query the Supplier Telephone List View
SELECT * FROM SupplierTelephoneList;
```



QUERY 2: INVOICES OVER £200

In the executed SQL question, I distinguished solicitations with an all out esteem surpassing £200 from the Receipt table. The SELECT assertion recovers all segments from the Receipt table where the total_value section surpasses the predefined limit. The WHERE proviso channels the records in view of the predefined condition, guaranteeing that main solicitations meeting the measure are remembered for the outcome set. The Request BY provision organizes the outcomes in slipping request of complete worth, introducing the most noteworthy esteemed solicitations first.

```
-- Query#2: to Identify Invoices Over £200
SELECT *
FROM Invoice
WHERE total_value > 200
ORDER BY total_value DESC;
```

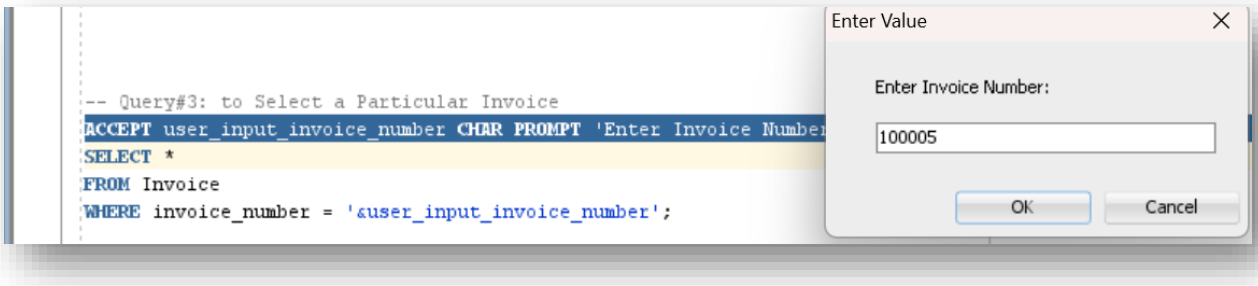


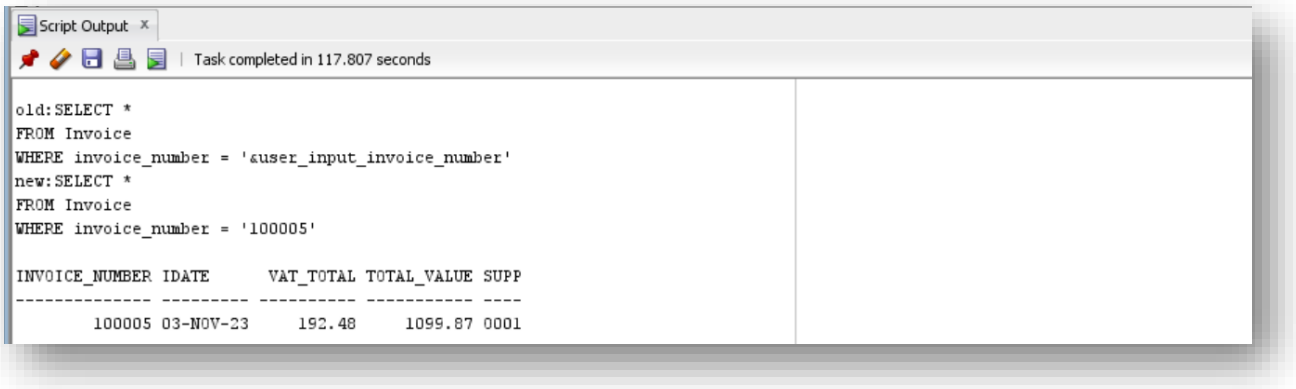
QUERY 3: SELECTION OF A PARTICULAR INVOICE

In the introduced SQL question, the point is to work with the choice of a particular receipt in light of client input. The question prompts the client to enter a receipt number, and afterward uses the Acknowledge articulation to accumulate this info. The resulting SELECT explanation recovers all sections from the Receipt table where the invoice_number matches the client gave input. The WHERE proviso guarantees that main the receipt with the predefined receipt number is remembered for the outcome set.

```
-- Query to Select a Particular Invoice
ACCEPT user_input_invoice_number CHAR PROMPT 'Enter Invoice Number: '

SELECT *
FROM Invoice
WHERE invoice_number = '&user_input_invoice_number';
```

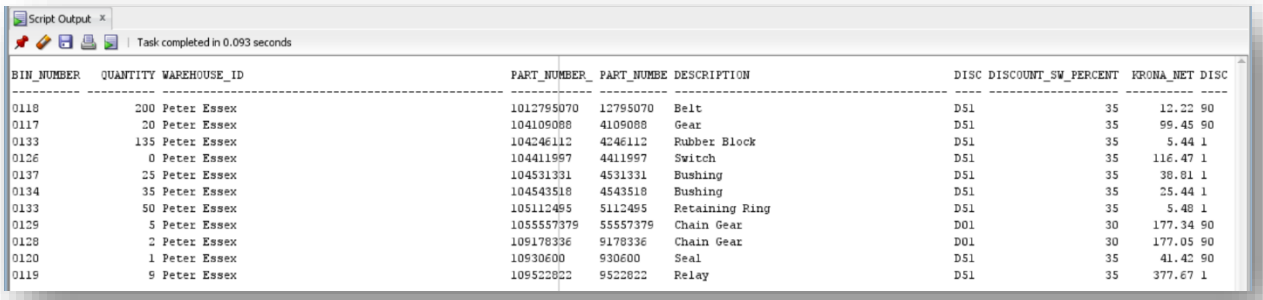




QUERY 4: LIST OF PARTS FOR BOTH WAREHOUSES

The gave SQL question plans to create a complete rundown of parts put away in the two stockrooms. It accomplishes this by joining applicable tables: Area, Container, Distribution center, and Part. The SELECT condition incorporates traits, for example, bin_number, amount, warehouse_id, and different qualities connected with parts, as part_number_with_prefix, depiction, and valuing subtleties. The Request BY proviso sorts out the outcome set first by distribution center ID in rising request and afterward by part number with prefix in climbing request.

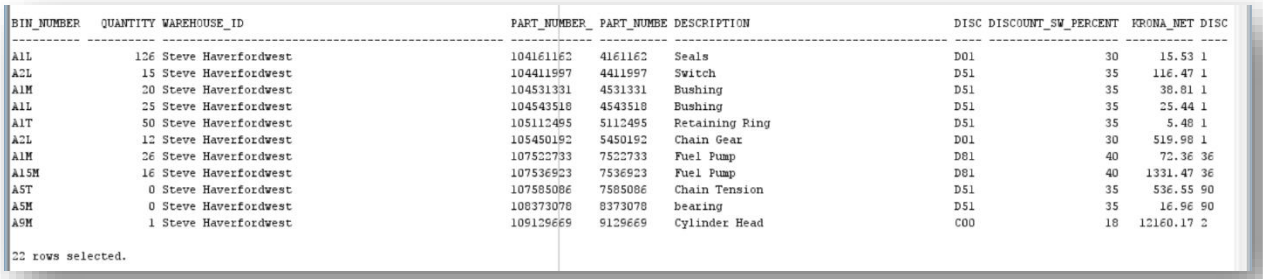
```
-- Query#4: List of parts for both warehouses
SELECT l.bin_number,
    l.quantity,
    w.warehouse_id,
    p.part_number_with_prefix,
    p.part_number,
    p.description,
    p.discount_sw_code,
    p.discount_sw_percent,
    p.krona_net,
    p.discount_uk_code,
    p.discount_uk_percent,
    p.sterling_net
FROM Location l
JOIN Bin b ON l.bin_number = b.bin_number
JOIN Warehouse w ON b.warehouse_id = w.warehouse_id
JOIN Part p ON l.part_number_with_prefix = p.part_number_with_prefix
ORDER BY w.warehouse_id ASC, p.part_number_with_prefix ASC;
```



Script Output

Task completed in 0.093 seconds

BIN_NUMBER	QUANTITY	WAREHOUSE_ID	PART_NUMBER_	PART_NUMBE	DESCRIPTION	DISC	DISCOUNT_SW_PERCENT	KRONA_NET	DISC
0118	200	Peter Essex	1012795070	12795070	Belt	D51	35	12.22	90
0117	20	Peter Essex	104109088	4109088	Gear	D51	35	99.45	90
0133	135	Peter Essex	104246112	4246112	Rubber Block	D51	35	5.44	1
0126	0	Peter Essex	104411997	4411997	Switch	D51	35	116.47	1
0137	25	Peter Essex	104531331	4531331	Bushing	D51	35	38.81	1
0134	35	Peter Essex	104543518	4543518	Bushing	D51	35	25.44	1
0133	50	Peter Essex	105112495	5112495	Retaining Ring	D51	35	5.48	1
0129	5	Peter Essex	1055557379	55557379	Chain Gear	D01	30	177.34	90
0128	2	Peter Essex	109178336	9178336	Chain Gear	D01	30	177.05	90
0120	1	Peter Essex	10930600	930600	Seal	D51	35	41.42	90
0119	9	Peter Essex	109522822	9522822	Relay	D51	35	377.67	1



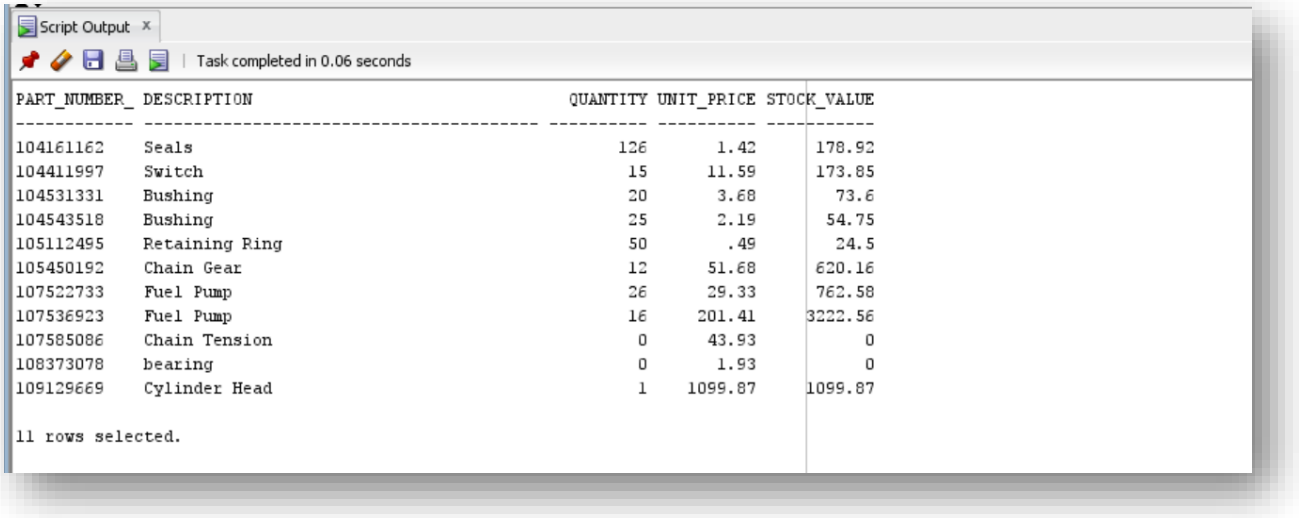
BIN_NUMBER	QUANTITY	WAREHOUSE_ID	PART_NUMBER_	PART_NUMBE	DESCRIPTION	DISC	DISCOUNT_SW_PERCENT	KRONA_NET	DISC
A1L	126	Steve Haverfordwest	104161162	4161162	Seals	D01	30	15.53	1
A2L	15	Steve Haverfordwest	104411997	4411997	Switch	D51	35	116.47	1
A1M	20	Steve Haverfordwest	104531331	4531331	Bushing	D51	35	38.81	1
A1L	25	Steve Haverfordwest	104543518	4543518	Bushing	D51	35	25.44	1
A1T	50	Steve Haverfordwest	105112495	5112495	Retaining Ring	D51	35	5.48	1
A2L	12	Steve Haverfordwest	105450192	5450192	Chain Gear	D01	30	519.98	1
A1M	26	Steve Haverfordwest	107522733	7522733	Fuel Pump	D81	40	72.36	36
A15M	16	Steve Haverfordwest	107536923	7536923	Fuel Pump	D81	40	1331.47	36
A5T	0	Steve Haverfordwest	107585086	7585086	Chain Tension	D51	35	536.55	90
A5M	0	Steve Haverfordwest	108373078	8373078	bearing	D51	35	16.96	90
A9M	1	Steve Haverfordwest	109129669	9129669	Cylinder Head	C00	18	12160.17	2

22 rows selected.

QUERY 5: LIST OF PARTS AND STOCK VALUE IN HAVERFORDWEST

The gave SQL question plans to produce a rundown of parts alongside their stock qualities explicitly in the Haverfordwest distribution center. To achieve this, the inquiry includes joining the Area, Canister, Distribution center, and Part tables, interfacing them in view of their separate keys. The SELECT statement incorporates credits like part_number_with_prefix, depiction, amount, unit_price (addressed by sterling_net), and stock_value (determined as the result of amount and unit cost). The WHERE statement channels the outcomes to just incorporate data connected with the Haverfordwest distribution center. The Request BY condition coordinates the result in view of the part number with prefix in rising request.

```
-- Query #5: List of parts and their stock values in Haverfordwest warehouse
SELECT l.part_number_with_prefix,
    p.description,
    l.quantity,
    p.sterling_net AS unit_price,
    l.quantity * p.sterling_net AS stock_value
FROM Location l
JOIN Bin b ON l.bin_number = b.bin_number
JOIN Warehouse w ON b.warehouse_id = w.warehouse_id
JOIN Part p ON l.part_number_with_prefix = p.part_number_with_prefix
WHERE w.warehouse_id = 'Steve Haverfordwest'
ORDER BY l.part_number_with_prefix ASC;
```



QUERY 6: VIEWS AND TOTAL STOCK VALUE IN HAVERFORDWEST

The gave SQL questions are centered around overseeing and examining what is happening in the Haverfordwest stockroom. How about we figure out down the code and its usefulness:

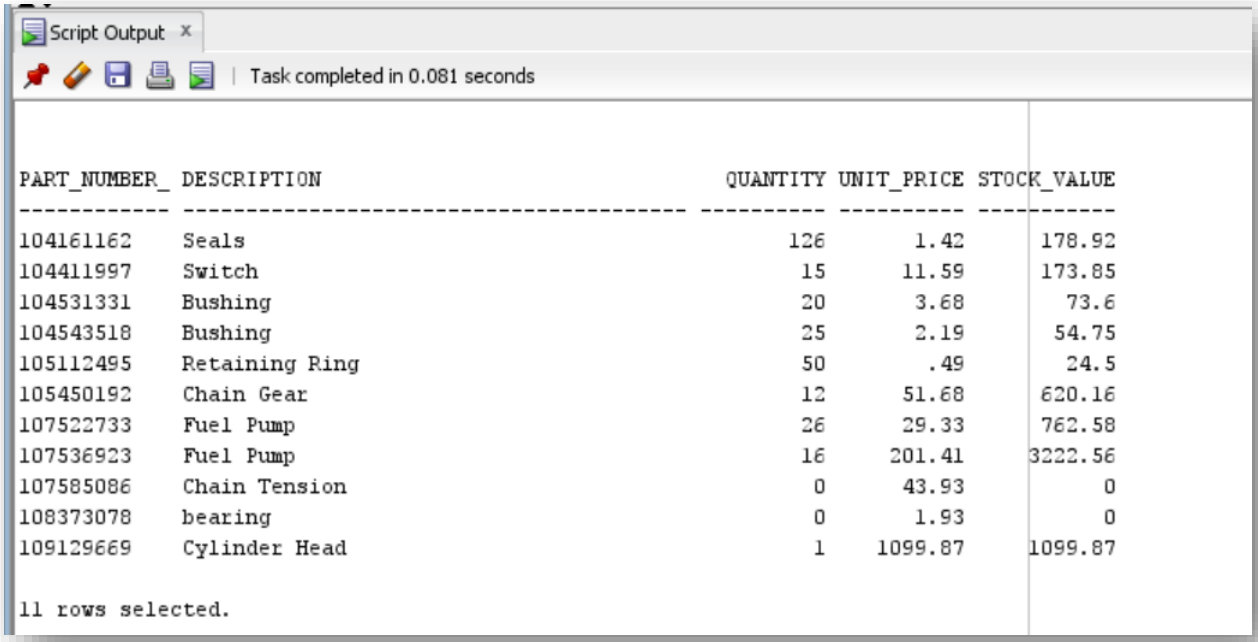
A view, first and foremost, named HaverfordwestStockView is made involving a similar rationale as Inquiry #5. This view solidifies data about parts, including their depictions, amounts, unit costs (addressed by sterling_net), and determined stock qualities, explicitly for the Haverfordwest distribution center.

Then, another view named HaverfordwestTotalStockValue is made. This view computes the complete stock worth in the Haverfordwest distribution center by summarizing the stock_value section from the HaverfordwestStockView.

At long last, a question recovers and shows the all out stock incentive for the Haverfordwest distribution center utilizing the HaverfordwestTotalStockValue view.

```
-- Create a view from Query #5
CREATE OR REPLACE VIEW HaverfordwestStockView AS
SELECT l.part_number_with_prefix,
    p.description,
    l.quantity,
    p.sterling_net AS unit_price,
    l.quantity * p.sterling_net AS stock_value
FROM Location l
JOIN Bin b ON l.bin_number = b.bin_number
JOIN Warehouse w ON b.warehouse_id = w.warehouse_id
JOIN Part p ON l.part_number_with_prefix = p.part_number_with_prefix
WHERE w.warehouse_id = 'Steve Haverfordwest';
-- Calculate the total value of all stock in Haverfordwest warehouse
CREATE OR REPLACE VIEW HaverfordwestTotalStockValue AS
```

```
SELECT SUM(stock_value) AS total_stock_value
FROM HaverfordwestStockView;
-- Query to get the total stock value for Haverfordwest warehouse
SELECT * FROM HaverfordwestTotalStockValue;
```

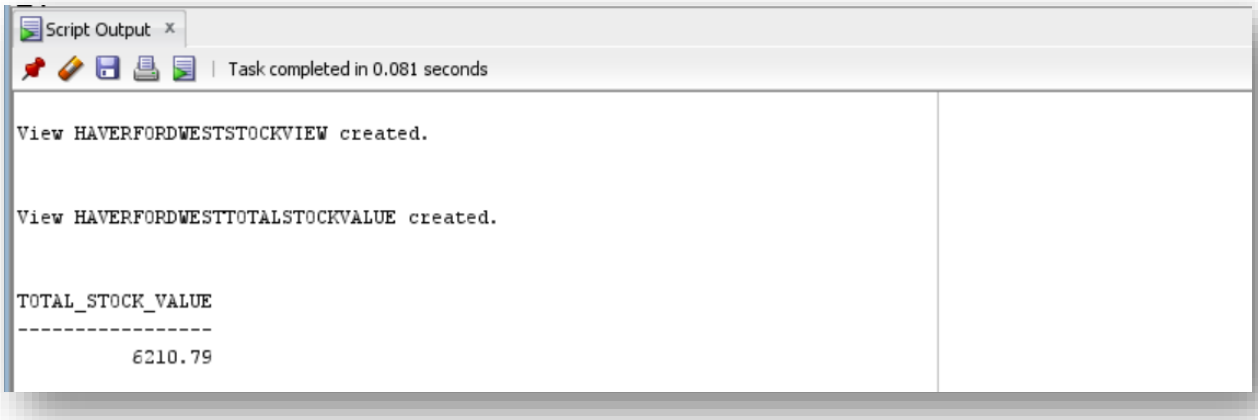


Script Output

Task completed in 0.081 seconds

PART_NUMBER_	DESCRIPTION	QUANTITY	UNIT_PRICE	STOCK_VALUE
104161162	Seals	126	1.42	178.92
104411997	Switch	15	11.59	173.85
104531331	Bushing	20	3.68	73.6
104543518	Bushing	25	2.19	54.75
105112495	Retaining Ring	50	.49	24.5
105450192	Chain Gear	12	51.68	620.16
107522733	Fuel Pump	26	29.33	762.58
107536923	Fuel Pump	16	201.41	3222.56
107585086	Chain Tension	0	43.93	0
108373078	bearing	0	1.93	0
109129669	Cylinder Head	1	1099.87	1099.87

11 rows selected.



Script Output

Task completed in 0.081 seconds

View HAVERFORDWESTSTOCKVIEW created.

View HAVERFORDWESTTOTALSTOCKVALUE created.

TOTAL_STOCK_VALUE
6210.79

QUERY 7: PARTS OUT OF STOCK IN BOTH WAREHOUSES

This question uses a subquery with the NOT EXISTS condition to sift through parts from the Part table that have no coordinating records in the Area table with the predefined conditions. The circumstances in the subquery check for the presence of records where the amount is more prominent than 0, demonstrating that the part is available, and the bin_number isn't invalid.

At the end of the day, the principal question recovers parts that have no relating records in the Area table where the part is available in any container in the two distribution centers.

```
SELECT p.part_number_with_prefix,
      p.description
FROM Part p
WHERE NOT EXISTS (
   SELECT 1
   FROM Location l
   WHERE l.part_number_with_prefix = p.part_number_with_prefix
     AND l.quantity > 0
     AND l.bin_number IS NOT NULL
);
```

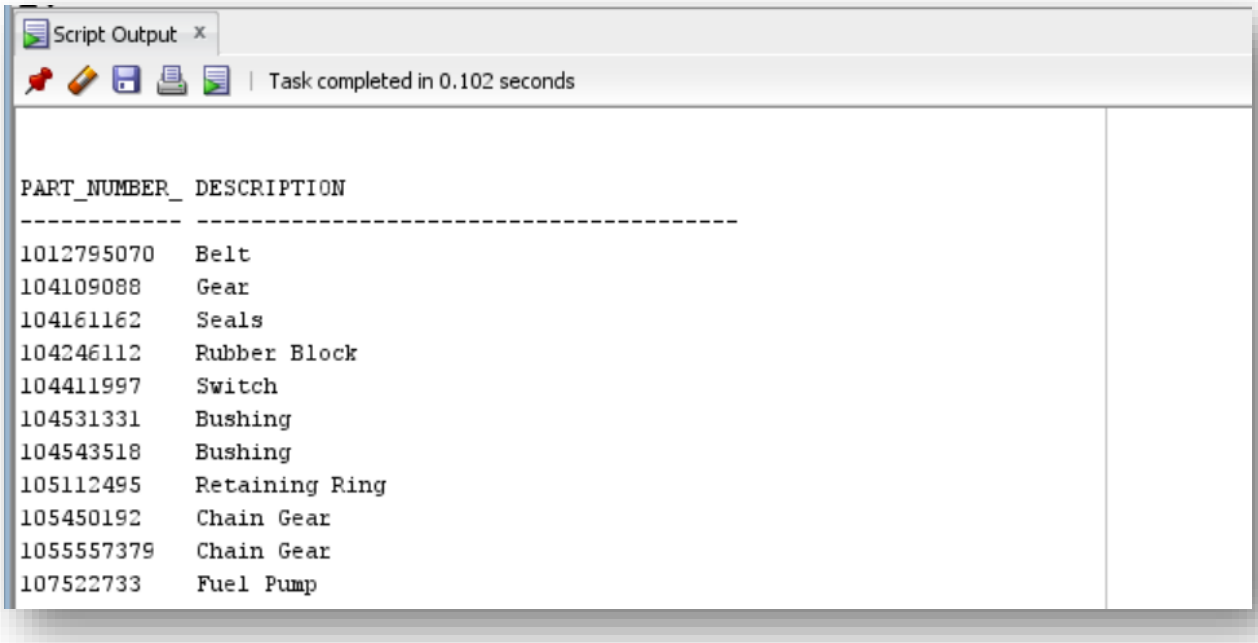
QUERY 8: PARTS AVAILABLE FROM BOTH WAREHOUSES

In the initial segment, a view named AvailableParts is made. This view incorporates the part_number_with_prefix from the Area table, sifting just those records where the amount is more prominent than 0, showing accessibility, and bin_number isn't invalid.

The ensuing questions then, at that point, utilize the EXISTS proviso to check for the presence of records in the AvailableParts view. The principal SELECT proclamation recovers parts that are accessible in no less than one distribution center, while the second SELECT assertion recovers parts accessible in the two stockrooms.

```
--Query#8
CREATE VIEW AvailableParts AS
SELECT part_number_with_prefix
FROM Location
WHERE quantity > 0 AND bin_number IS NOT NULL;
SELECT p.part_number_with_prefix,
      p.description
```

```
FROM Part p
WHERE EXISTS (
   SELECT 1
   FROM AvailableParts ap
   WHERE ap.part_number_with_prefix = p.part_number_with_prefix
);
SELECT p.part_number_with_prefix,
     p.description
FROM Part p
WHERE EXISTS (
   SELECT 2
   FROM AvailableParts ap
   WHERE ap.part_number_with_prefix = p.part_number_with_prefix
);
```



QUERY 9: PARTS CHEAPER TO BUY FROM SWEDEN

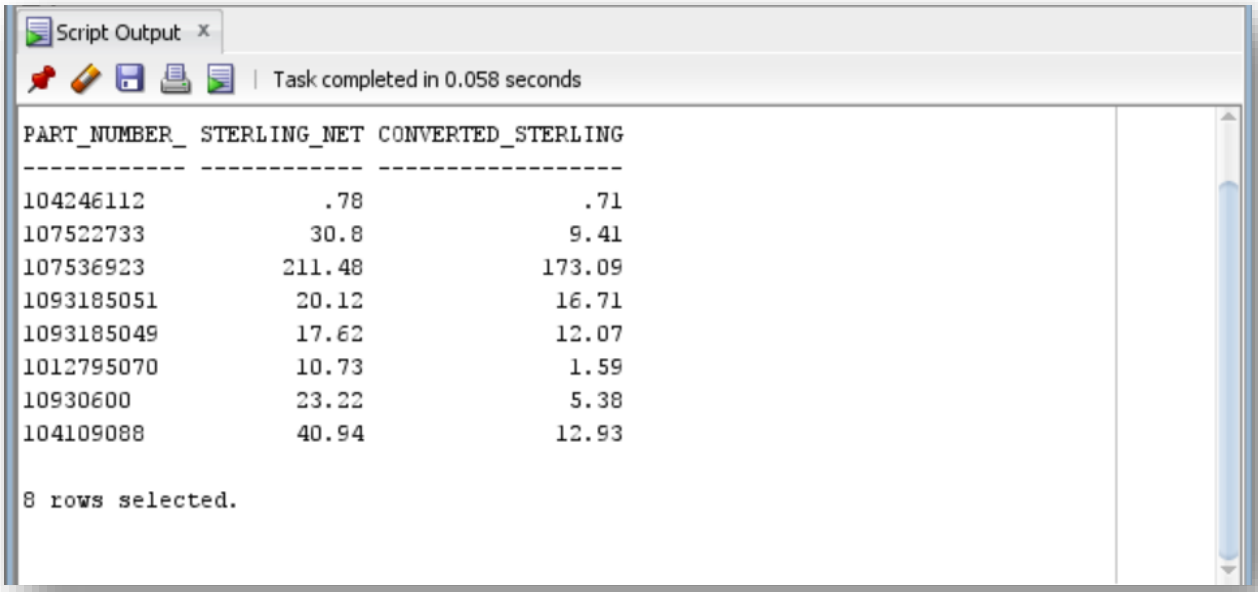
This query joins the Part table with the Supplier table using the condition that the discount code for Sweden (discount_sw_code) matches the supplier number. The selected columns include the part_number_with_prefix, the original price in Krona (sterling_net), the converted price to Sterling (using a conversion factor of 13%), and a case statement to determine whether the part is cheaper in Sterling, Krona, or if the prices are equal.

This next query is from the context of a system that seems to have a Part table containing information about different parts, including their part_number_with_prefix, original price in Sterling (sterling_net), and the corresponding price in Krona (krona_net).

The query calculates the converted price to Sterling using the formula krona_net * (13/100) and rounds the result to 2 decimal places with the ROUND function. The WHERE clause filters the results to only include rows where the converted Sterling price is less than the original Sterling price. This implies that the parts are cheaper when purchased in Krona and converted to Sterling.

```
--Query#9
SELECT
   p.part_number_with_prefix,
   p.sterling_net AS krona_net_in_sterling,
   ROUND((p.sterling_net * 13 / 100), 2) AS converted_to_sterling,
   CASE
      WHEN ROUND((p.sterling_net * 13 / 100), 2) < p.sterling_net THEN 'Cheaper in Sterling'
      WHEN ROUND((p.sterling_net * 13 / 100), 2) > p.sterling_net THEN 'Cheaper in Krona'
      ELSE 'Equal in Both Currencies'
   END AS cheaper_currency
FROM
   Part p
INNER JOIN
   Supplier s ON p.discount_sw_code = s.supplier_number
WHERE
   s.country = 'Sweden';
```

```
SELECT part_number_with_prefix, sterling_net, ROUND(krona_net * (13/100), 2) AS
converted_sterling
FROM Part
WHERE ROUND(krona_net * (13/100), 2) < sterling_net;
```



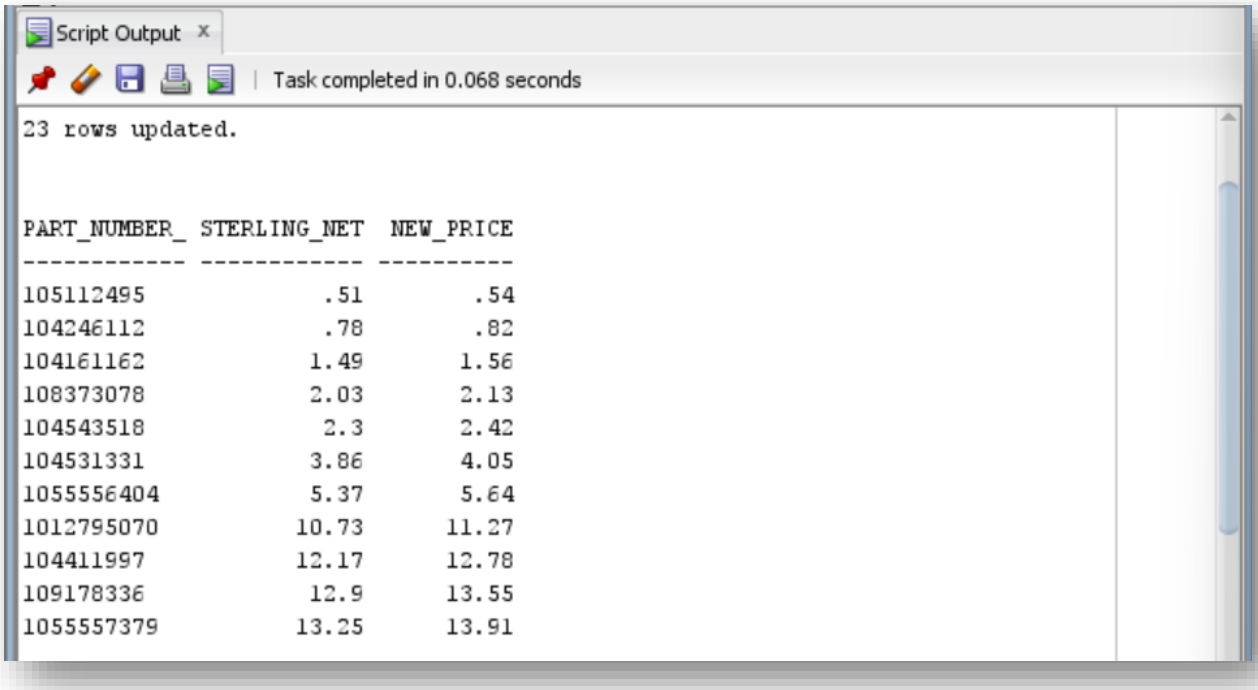
QUERY 10: INCREASE THE COST OF ALL PARTS BY 5%

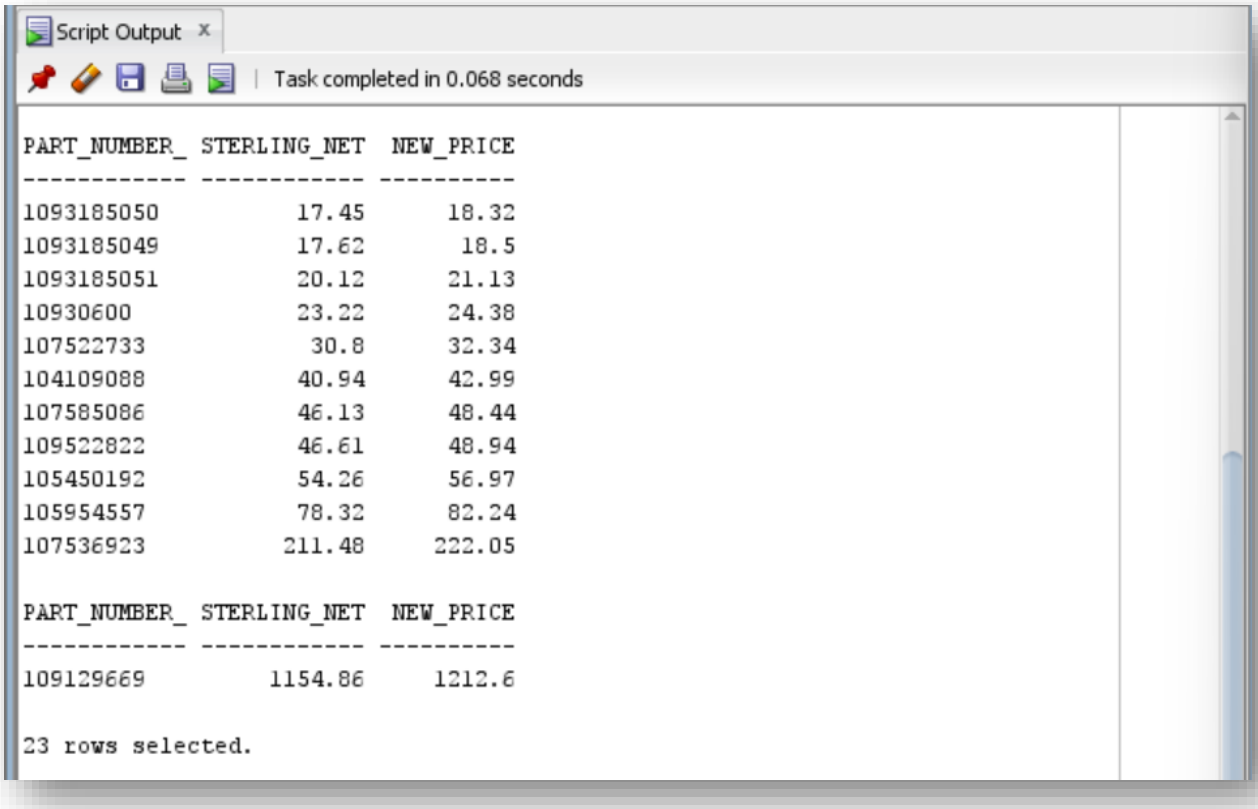
The UPDATE statement modifies the sterling_net column of all rows in the Part table by multiplying the current sterling_net value by 1.05 (increasing it by 5%). The ROUND function is applied to round the result to 2 decimal places, ensuring precision in the updated values.

The subsequent SELECT statement retrieves the part_number_with_prefix, original sterling_net, and the updated sterling_net (calculated as the original price multiplied by 1.05) for all parts in the table. The ORDER BY new_price clause is used to display the results in ascending order based on the updated Sterling prices.

```
UPDATE Part
SET sterling_net = ROUND(sterling_net * 1.05, 2);

SELECT part_number_with_prefix, sterling_net, ROUND(sterling_net * 1.05, 2) AS new_price
FROM Part
ORDER BY new_price;
```


Script Output
Task completed in 0.068 seconds
23 rows updated.
PART_NUMBER_ STERLING_NET NEW_PRICE
105112495 .51 .54
104246112 .78 .82
104161162 1.49 1.56
108373078 2.03 2.13
104543518 2.3 2.42
104531331 3.86 4.05
1055556404 5.37 5.64
1012795070 10.73 11.27
104411997 12.17 12.78
109178336 12.9 13.55
1055557379 13.25 13.91


Script Output
Task completed in 0.068 seconds
PART_NUMBER_ STERLING_NET NEW_PRICE
1093185050 17.45 18.32
1093185049 17.62 18.5
1093185051 20.12 21.13
10930600 23.22 24.38
107522733 30.8 32.34
104109088 40.94 42.99
107585086 46.13 48.44
109522822 46.61 48.94
105450192 54.26 56.97
105954557 78.32 82.24
107536923 211.48 222.05
PART_NUMBER_ STERLING_NET NEW_PRICE
109129669 1154.86 1212.6
23 rows selected.

APPENDIX:

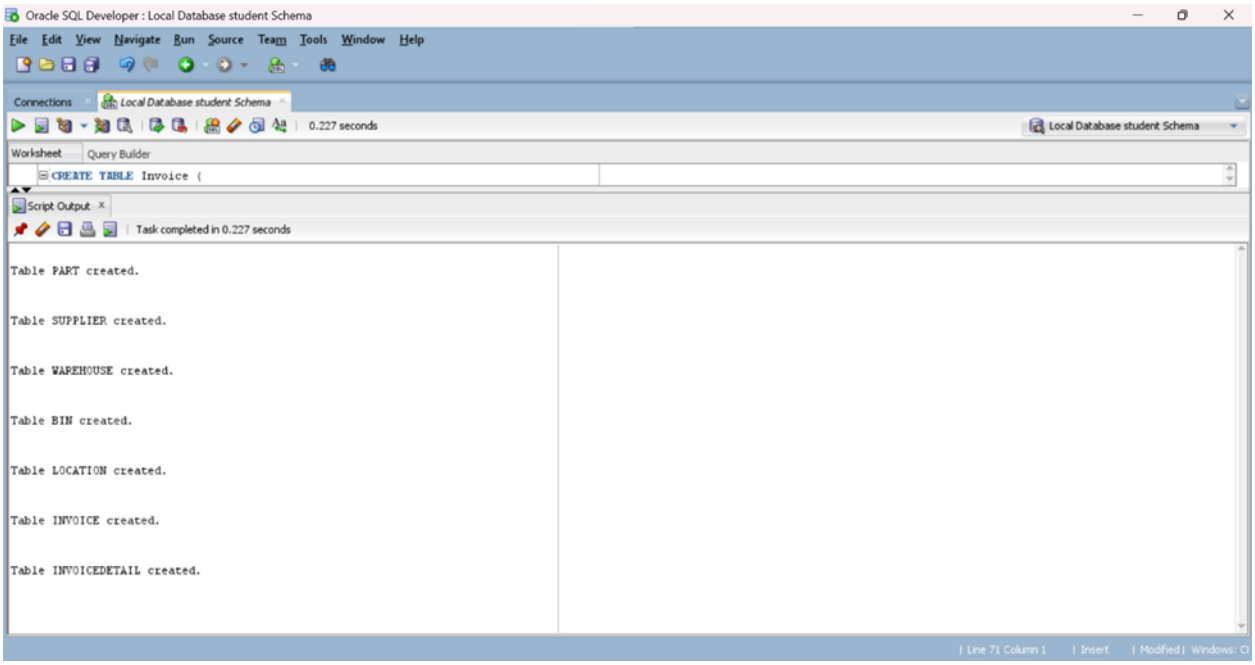
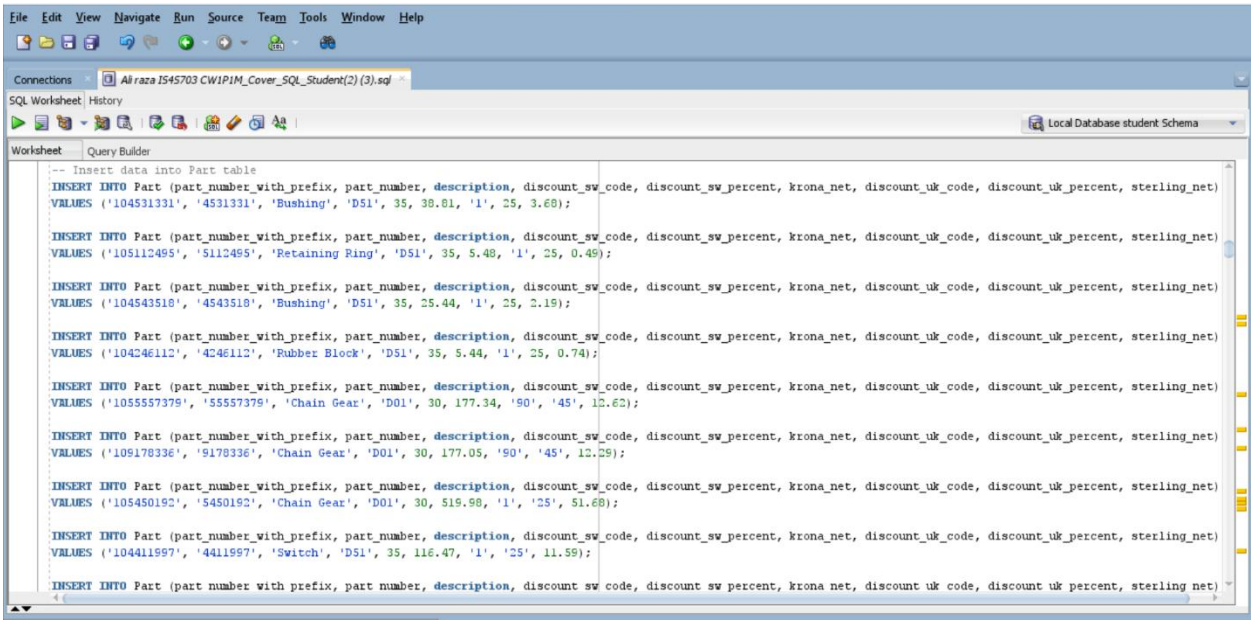


FIGURE 3:SUCCESSFULL TABLES CREATION:OUTPUT SCREEN



```
-- Inserting data into the Supplier table
INSERT INTO Supplier (supplier_number, supplier_name, address, town, county, postcode, contact_number, email_address, country)
VALUES ('H001', 'Higher Oak', 'Oak Road, Wrexham Ind Est', 'Wrexham', 'Wrexham', 'LL13 9RG', '01443456123', 'HighOak@hotmail.co.uk', 'Wales');

INSERT INTO Supplier (supplier_number, supplier_name, address, town, county, postcode, contact_number, email_address, country)
VALUES ('N001', 'Nordic Car Company', 'Unit 2, ByFleet Technical Centre', 'ByFleet', 'Surrey', 'KT14 7JL', '01798445566', 'NordicCars@tiscali.co.uk', 'Great Britain');

INSERT INTO Supplier (supplier_number, supplier_name, address, town, county, postcode, contact_number, email_address, country)
VALUES ('S002', 'Swain and Jones', '35-42 East Street', 'Farnham', 'Surrey', 'GU9 7SW', '01798776589', 'French1@hotmail.co.uk', 'Wales');

INSERT INTO Supplier (supplier_number, supplier_name, address, town, county, postcode, contact_number, email_address, country)
VALUES ('U001', 'Ultimate Car Force', 'Unit 4, Burrows Ind Est', 'Shere, Guildford', 'Surrey', 'GU55 9QQ', '01567836425', 'UltimateCars@yahoo.com', 'Great Britain');

INSERT INTO Supplier (supplier_number, supplier_name, address, town, county, postcode, contact_number, email_address, country)
VALUES ('W001', 'Workshop Saab Specialists', '9 Clothier Road', 'Brislington', 'Bristol', 'BS4 5PS', '01792623594', 'TheWorkshop@btconnect.com', 'Great Britain');

INSERT INTO Supplier (supplier_number, supplier_name, address, town, county, postcode, contact_number, email_address, country)
VALUES ('A001', 'Abbot Racing', 'Spinnels Farm, Wix', 'Manningtree', 'Essex', 'CO11 2UJ', '01279641225', 'AbbottRacing@yahoo.co.uk', 'Great Britain');

INSERT INTO Supplier (supplier_number, supplier_name, address, town, county, postcode, contact_number, email_address, country)
VALUES ('A002', 'Autohaus Furst GMBH', 'Berbillger Str 4, 71254 Ditzingen', NULL, NULL, NULL, '0049711918931', 'Autohaus@yahoo.de', 'Germany');

INSERT INTO Supplier (supplier_number, supplier_name, address, town, county, postcode, contact_number, email_address, country)
VALUES ('B001', 'Bond Street', 'Kerry House, 108 Vaughan Way', 'Leicester', 'Leicester', 'LE2 6HJ', '01664765533', 'BondStreetSaab@hotmail.co.uk', 'Great Britain');
```

FIGURE 4: DATA POPULATE EVIDENCE